

实 验 报 告

实验名称： 实验四 LSTM 的实现

课程名称： 认知科学与类脑计算

实验地点： K2 227

实验日期： 2025/9/12

班 级： 23 级智能班

姓 名： 宋浩宇

学 号： 202300130183

一 实验目的：

加深对 LSTM 模型的理解，能够使用 LSTM 模型解决简单问题

二 实验环境：

主系统：Windows 11 家庭中文版 23H2 ; ArchLinux

编辑器：Visual Studio Code ; PyCharm 2025.1.2 ; neovim 0.11

Python 解释器：Python 3.9

Pytorch 版本：2.8.0

CUDA 版本：12.8.97

硬件环境：

CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz

内存：32.0 GB (31.6 GB 可用)

磁盘驱动器：NVMe WD_BLACKSN850X2000GB

显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU

CUDA 核心数：7424

三 实验内容：

根据 LSTM 模型的相关知识，使用 Python 语言实现一个简单 LSTM 模型。

四 实验步骤：

1. 实验程序：

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from tqdm import tqdm

# 工具函数
def int_to_bin_array(n, width=8):
    return np.array([int(b) for b in format(n, f'0{width}b')[::-1]],
dtype=np.float32)
def bin_array_to_int(arr):
    return int(''.join(str(int(x)) for x in arr[::-1]), 2)
def get_random_pair():
    n1 = np.random.randint(0, 128)
    n2 = np.random.randint(0, 128)
    return (n1, n2), n1 + n2
# 普通整数加法数据集
def generate_dataset_int(num_samples=2000):
    dataset = []
```

```

        for _ in tqdm(range(num_samples), desc="Generating integer
dataset"):
            (n1, n2), label = get_random_pair()
            dataset.append((n1, n2), label))
        return dataset
# 二进制序列加法数据
def generate_dataset_binary(num_samples=2000):
    dataset = []
    for _ in tqdm(range(num_samples), desc="Generating binary
dataset"):
        (n1, n2), label = get_random_pair()
        x1 = int_to_bin_array(n1)
        x2 = int_to_bin_array(n2)
        y = int_to_bin_array(label % 256) # 限制为 8 位结果
        # 输入 shape: (seq_len=8, input_size=2)
        x = np.stack([x1, x2], axis=1)
        dataset.append((x, y))
    return dataset
# LSTM Cell
class LSTMCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        # 参数定义
        self.W_f = nn.Parameter(torch.randn(input_size + hidden_size,
hidden_size) * 0.1)
        self.b_f = nn.Parameter(torch.zeros(hidden_size))
        self.W_i = nn.Parameter(torch.randn(input_size + hidden_size,
hidden_size) * 0.1)
        self.b_i = nn.Parameter(torch.zeros(hidden_size))
        self.W_c = nn.Parameter(torch.randn(input_size + hidden_size,
hidden_size) * 0.1)
        self.b_c = nn.Parameter(torch.zeros(hidden_size))
        self.W_o = nn.Parameter(torch.randn(input_size + hidden_size,
hidden_size) * 0.1)
        self.b_o = nn.Parameter(torch.zeros(hidden_size))
    def forward(self, x, h_prev, c_prev):
        combined = torch.cat((x, h_prev), dim=1)
        f = torch.sigmoid(combined @ self.W_f + self.b_f)
        i = torch.sigmoid(combined @ self.W_i + self.b_i)
        c_hat = torch.tanh(combined @ self.W_c + self.b_c)
        c = f * c_prev + i * c_hat
        o = torch.sigmoid(combined @ self.W_o + self.b_o)

```

```

        h = o * torch.tanh(c)
        return h, c
# 整数加法模型
class LSTMNet4Int(nn.Module):
    def __init__(self, input_size=2, hidden_size=32):
        super().__init__()
        self.hidden_size = hidden_size
        self.cell = LSTMCell(input_size, hidden_size)
        self.fc = nn.Linear(hidden_size, 1)
    def forward(self, x):
        batch_size, seq_len, _ = x.shape
        h = torch.zeros(batch_size, self.hidden_size,
device=x.device)
        c = torch.zeros(batch_size, self.hidden_size,
device=x.device)
        for t in range(seq_len):
            h, c = self.cell(x[:, t, :], h, c)
        return self.fc(h).squeeze(1)
# 二进制序列加法模型
class LSTMNet4Binary(nn.Module):
    def __init__(self, input_size=2, hidden_size=32, output_size=8):
        super().__init__()
        self.hidden_size = hidden_size
        self.cell = LSTMCell(input_size, hidden_size)
        self.fc = nn.Linear(hidden_size, 1) # 每位输出一个 bit
    def forward(self, x):
        batch_size, seq_len, _ = x.shape
        h = torch.zeros(batch_size, self.hidden_size,
device=x.device)
        c = torch.zeros(batch_size, self.hidden_size,
device=x.device)
        outputs = []
        for t in range(seq_len):
            h, c = self.cell(x[:, t, :], h, c)
            out_t = torch.sigmoid(self.fc(h)) # 每步输出一个 bit(0~1)
            outputs.append(out_t)
        return torch.cat(outputs, dim=1) # (batch, seq_len)
# 训练函数
def train_model(model, dataset, epochs=100, lr=0.001, batch_size=32,
binary=False):
    device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
    model.to(device)
    print(f"训练设备: {device}")

```

```

optimizer = optim.Adam(model.parameters(), lr=lr)
criterion = nn.MSELoss()
print("\n 开始训练...")
for epoch in range(epochs):
    np.random.shuffle(dataset)
    total_loss = 0.0
    batch_iter = tqdm(range(0, len(dataset), batch_size),
desc=f"Epoch {epoch + 1}/{epochs}", leave=True)
    for i in batch_iter:
        batch = dataset[i:i + batch_size]
        xs = np.array([item[0] for item in batch],
dtype=np.float32)
        ys = np.array([item[1] for item in batch],
dtype=np.float32)
        if not binary:
            xs = np.expand_dims(xs, axis=1) # (batch, 1, 2)
            xs = torch.tensor(xs, dtype=torch.float32).to(device)
            ys = torch.tensor(ys, dtype=torch.float32).to(device)
            optimizer.zero_grad()
            pred = model(xs)
            loss = criterion(pred, ys)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            batch_iter.set_postfix(loss=loss.item())
        avg_loss = total_loss / (len(dataset) / batch_size)
        tqdm.write(f"Epoch {epoch+1}/{epochs} - 平均 Loss:
{avg_loss:.6f}")
# 测试函数
def test_binary(model, num_tests=5):
    model.eval()
    device = next(model.parameters()).device
    print("\n 测试二进制加法预测:")
    with torch.no_grad():
        for _ in tqdm(range(num_tests), desc="Testing"):
            (n1, n2), label = get_random_pair()
            x1 = int_to_bin_array(n1)
            x2 = int_to_bin_array(n2)
            x = np.stack([x1, x2], axis=1)
            x_tensor = torch.from_numpy(np.array([x],
dtype=np.float32)).to(device)
            pred = model(x_tensor).cpu().numpy().squeeze()
            pred_bits = (pred > 0.5).astype(int)
            pred_int = bin_array_to_int(pred_bits)

```

```

        print(f"{n1} + {n2} = {label:3d} | 预测二进制:
{pred_bits[:::-1]} => {pred_int}")
def test_int(model, num_tests=5):
    model.eval()
    device = next(model.parameters()).device
    with torch.no_grad():
        for _ in range(num_tests):
            (n1, n2), label = get_random_pair()
            x = torch.tensor([[[n1, n2]]],
dtype=torch.float32).to(device)
            pred = model(x).item()
            print(f"{n1} + {n2} = {label}, 预测: {pred:.2f}")
if __name__ == "__main__":
    # 普通整数加法
    dataset_int = generate_dataset_int(3000)
    model_int = LSTMNet4Int(input_size=2, hidden_size=64)
    train_model(model_int, dataset_int, epochs=50, lr=0.001,
binary=False)
    test_int(model_int)
    print("\n" + "="*60 + "\n")
    # 二进制序列加法
    dataset_bin = generate_dataset_binary(4000)
    model_bin = LSTMNet4Binary(input_size=2, hidden_size=64)
    train_model(model_bin, dataset_bin, epochs=50, lr=0.001,
binary=True)
    test_binary(model_bin)

```

2. 程序实验结果:

因为我没搞明白是让实现两个数字相加,还是两个二进制序列相加,所以两个我都写了一下,结果分别是:

```

11 + 29 = 40, 预测: 40.73
22 + 107 = 129, 预测: 129.23
57 + 0 = 57, 预测: 55.71
75 + 65 = 140, 预测: 144.83
59 + 95 = 154, 预测: 150.92

```

```

测试二进制加法预测:
72 + 28 = 100 | 预测二进制: [0 1 1 0 0 1 0 0] => 100
79 + 104 = 183 | 预测二进制: [1 0 1 1 0 1 1 1] => 183
38 + 107 = 145 | 预测二进制: [1 0 0 1 0 0 0 1] => 145
Testing: 100%|██████████| 5/5 [00:00<00:00, 412.47it/s]
80 + 108 = 188 | 预测二进制: [1 0 1 1 1 1 0 0] => 188
99 + 37 = 136 | 预测二进制: [1 0 0 0 1 0 0 0] => 136

```

这个 `tqdm` 的进度条是标准输出两个不同线程抢占导致的，请忽略

3. 分析和改进：

从结果上来看，得到的模型对数字相加还会有一个较小的误差，二从二进制上来看，结果已经能够比较正确了。改进的话，可以使用更复杂的改进后的 `LSTM` 的架构，或者调整一下损失函数，优化器，还有超参数。

五 实验小结：

从结果上来看得到的模型可以较好地完成这个任务。需要说明的是，我在完成这个实验的时候使用了 `pytorch`，不过并没有使用它提供的预设 `lstm` 模型，而自行实现的 `lstm` 架构，只是借助 `pytorch` 进行梯度下降。